

## Abstract

The report describes a MIPS assembler program written in the second programming exercise of the COMP 451 Compiler Construction course. The primary goal of this program is to take an input file containing a MIPS assembly language code and convert every instruction to a 32 bit machine code in hexadecimal form. The assembler is implemented using the C language and supports twelve commonly used MIPS instructions which are add, and, or, xor, slt, addi, andi, ori, lw, sw, beq and j.

This assembler follows a two pass approach for accurate translation. In the first pass, the program scans all the lines of the input file to identify labels and stores them along with their corresponding memory addresses in a symbol table. This step helps so that all label references can be resolved correctly in later stages. In the second pass, the program processes each instruction line by line and then converts it into machine code using the appropriate instruction format. It correctly handles R-type, I-type and J-type instructions by applying the required opcode, register values, immediate fields and function codes.

The assembler output is presented in a tabular format made up of three columns namely the memory address, the generated hexadecimal machine code and the original assembly code. This facilitates easier verification and tracking of the process of translation.

In order to ensure the correctness and reliability of the program I utilised six different input files for testing purposes. Each of the twelve supported instructions appears in at least two of these test files to provide sufficient coverage. The results produced by the assembler were carefully verified by comparing them with the output generated by the MARS MIPS simulator. All outputs matched exactly and this confirms that the assembler is functioning correctly and producing accurate machine code translations.

## Introduction

When a programmer writes code in assembly language, the computer does not understand it directly. The computer only understands binary numbers which are ones and zeros. An assembler is a program that sits in the middle and translates assembly language into binary machine code that the processor can actually run. This is why assemblers are an important part of the study of compilers.

MIPS stands for Microprocessor without Interlocked Pipeline Stages. It is a type of processor architecture that is very popular in computer science education because its instruction set is simple and clean. Each MIPS instruction is exactly 32 bits long. MIPS instructions are grouped into three formats which are called R-type, I-type, and J-type. Each format arranges the 32 bits in a specific way.

In this assignment the task was to write a MIPS assembler in C that can handle twelve specific instructions. The main question this project tries to answer is: how can a C program correctly read MIPS assembly instructions and produce the exact same 32-bit machine code that a real MIPS processor would use?

The answer to this question was found by studying the MIPS instruction encoding format, understanding how bits are arranged in each instruction type and implementing the correct bitwise operations in C. The program was then tested against the MARS MIPS simulator which is a standard tool used in computer architecture courses to verify MIPS assembly programs.

# Literature Review

## MIPS Instruction Formats

According to standard MIPS processor documentation, every MIPS instruction is exactly 32 bits wide. These 32 bits are divided into different fields depending on the instruction type. There are three instruction types in the MIPS architecture.

R-type instructions are used when all operands are registers. The 32 bits are divided into six fields: opcode (6 bits), rs (5 bits), rt (5 bits), rd (5 bits), shamt (5 bits), and funct (6 bits). For all R-type instructions the opcode is always zero. The funct field is used to identify which specific operation is being done. The rs and rt fields are the source registers and rd is the destination register.

I-type instructions are used when one operand is an immediate value which is a number written directly in the instruction. The 32 bits are divided into four fields: opcode (6 bits), rs (5 bits), rt (5 bits), and immediate (16 bits). The opcode identifies the instruction. The rs is the source register, rt is the destination or second register, and the immediate is a signed 16-bit number.

J-type instructions are used for jump operations. The 32 bits are divided into two fields: opcode (6 bits) and target (26 bits). The target is the word address of the instruction to jump to.

| Type   | opcode (6) | rs (5)                   | rt (5) | rd (5)         | shamt+funct   |
|--------|------------|--------------------------|--------|----------------|---------------|
| R-type | 000000     | rs                       | rt     | rd             | shamt   funct |
| I-type | opcode     | rs                       | rt     | immediate (16) | —             |
| J-type | opcode     | target address (26 bits) | —      | —              | —             |

## BEQ Branch Offset Formula

For the beq instruction the immediate field stores a relative offset not an absolute address. The formula to calculate this offset is shown below. This formula was verified by testing with MARS.

$$\text{offset} = (\text{label\_address} - (\text{current\_PC} + 4)) / 4$$

The reason we add 4 to the current PC is that by the time the processor checks the branch condition it has already moved to the next instruction. The division by 4 is because MIPS addresses are byte addresses but the offset is counted in words and each word is 4 bytes. This

was verified: for beq at 0x400008 jumping to end at 0x400014,  $\text{offset} = (0x400014 - 0x40000C) / 4 = 2$ , giving machine code 0x11890002 which matches MARS.

### **J-Type Jump Target Formula**

For the j instruction the 26-bit target field stores the word address of the jump destination. The formula is:

$$\text{target} = (\text{label\_address} \gg 2) \text{ AND } 0x03FFFFFF$$

Shifting right by 2 converts a byte address to a word address. The mask keeps only the lower 26 bits. For j start where start is at 0x00400000, the target is 0x100000 and the machine code is 0x08100000 which matches MARS.

### **Two Pass Assembly**

The technique of reading the input file twice is called two pass assembly. This technique is needed because of forward references. A forward reference happens when an instruction uses a label that is defined later in the code. For example if beq jumps to a label called end but end is defined at the bottom of the file, the assembler needs to know the address of end before it has seen it. The first pass solves this by going through all lines first and collecting all label addresses. Then in the second pass the assembler already knows all label addresses and can calculate branch offsets correctly.

### **MARS MIPS Simulator**

MARS stands for MIPS Assembler and Runtime Simulator. It was developed at Missouri State University. MARS is a Java program that assembles and runs MIPS programs. It shows the assembled machine code in its Text Segment window. In this project MARS was used as the reference to verify that the machine code produced by our assembler is correct. Every output from our program was compared with the MARS Text Segment and all values matched.

## Methods

This section explains step by step everything that was done to build the assembler.

### Step 1 — Learning MIPS Instruction Encoding

The first step was to study the MIPS instruction formats and find the correct opcode and funct values for all twelve instructions. These values were confirmed by running tests in MARS. The complete table is below:

| Instruction | Type   | Opcode | Funct | Operand Format     |
|-------------|--------|--------|-------|--------------------|
| add         | R-type | 0x00   | 0x20  | \$rd, \$rs, \$rt   |
| and         | R-type | 0x00   | 0x24  | \$rd, \$rs, \$rt   |
| or          | R-type | 0x00   | 0x25  | \$rd, \$rs, \$rt   |
| xor         | R-type | 0x00   | 0x26  | \$rd, \$rs, \$rt   |
| slt         | R-type | 0x00   | 0x2A  | \$rd, \$rs, \$rt   |
| addi        | I-type | 0x08   | —     | \$rt, \$rs, imm    |
| andi        | I-type | 0x0C   | —     | \$rt, \$rs, imm    |
| ori         | I-type | 0x0D   | —     | \$rt, \$rs, imm    |
| lw          | I-type | 0x23   | —     | \$rt, offset(\$rs) |
| sw          | I-type | 0x2B   | —     | \$rt, offset(\$rs) |
| beq         | I-type | 0x04   | —     | \$rs, \$rt, label  |
| j           | J-type | 0x02   | —     | label              |

### Step 2 — Setting Up the C Program

The assembler program was written in a single C file called `obil.c`. The program was written and tested on Ubuntu Linux inside Oracle VirtualBox. Compilation was done with: `gcc obil.c -o test`. Running was done with: `./test in1.txt`

### Step 3 — Code

Below is a complete explanation of every function and section of the code written in simple language.

### 3.1 Header Files and Constants

```
#include <stdio.h>      /* for printf, fopen, fgets, fclose */
#include <stdlib.h>      /* for atoi - convert string to integer */
#include <string.h>      /* for strcmp, strcpy, strlen, strncpy */
#include <ctype.h>       /* for isdigit - check if char is digit */

#define MAX_LINES      1000          /* max lines program can read */
#define MAX_LINE_LEN   256           /* max characters per line */
#define BASE_ADDRESS   0x00400000    /* MIPS programs start here */
```

These are the standard C libraries the program needs. `stdio.h` is for reading files and printing. `stdlib.h` gives `atoi` which converts text like "16" into the number 16. `string.h` gives functions to compare and copy strings. `ctype.h` gives `isdigit` to check if a character is a number.

The base address `0x00400000` is the standard starting address for MIPS programs in MARS. Every instruction address starts here and increases by 4 for each instruction because each MIPS instruction is 4 bytes long.

### 3.2 Label Structure and Table

```
typedef struct {
    char name[64]; /* the label name e.g. start, loop, end */
    int address;   /* the memory address this label maps to */
} Label;

Label labelTable[200]; /* can store up to 200 labels */
int labelCount = 0; /* how many labels found so far */
```

A label in MIPS assembly is a name that marks a specific position in the code, for example `start`: or `loop`: or `end`:. When the program sees `beq` or `j` it needs to know the memory address that the label refers to. The `labelTable` stores all label names and their addresses. `labelCount` tracks how many labels have been found.

### 3.3 getRegNum — Register Name to Number

```
int getRegNum(char *reg) {
    char *r = reg;
    if (r[0] == '$') r++; /* skip the dollar sign */
    if (strcmp(r, "zero") == 0) return 0;
    if (strcmp(r, "t0") == 0) return 8;
    if (strcmp(r, "t1") == 0) return 9;
    /* all 32 registers listed this way */
    if (isdigit((unsigned char)r[0])) return atoi(r);
    return -1; /* unknown register */
}
```

MIPS has 32 registers numbered 0 to 31. In assembly code we write register names like `$t0`, `$s1`, `$sp`, `$zero`. But in machine code we need the register number not the name. This function does that conversion. It removes the dollar sign then checks which register it is and returns the correct

number. For example \$t0 becomes 8, \$t1 becomes 9, \$sp becomes 29, \$zero becomes 0. If the register is already a number like \$9 it uses atoi to convert directly.

### 3.4 trim — Removing Extra Whitespace

```
void trim(char *s) {
    int i = 0;
    while (s[i]==' ' || s[i]=='\t') i++;
    if (i > 0) memmove(s, s+i, strlen(s)-i+1);
    int len = strlen(s);
    while (len>0 && (s[len-1]==' ' || s[len-1]=='\t' ||
                    s[len-1]=='\n' || s[len-1]=='\r'))
        s[--len] = '\0';
}
```

When fgets reads a line from a file it includes the newline character at the end. Sometimes lines also have extra spaces at the beginning or end. This function removes all that extra whitespace from both sides of the string. This is important because if a line has a trailing newline character it would break string comparison functions like strcmp.

### 3.5 isLabel — Detecting Label Lines

```
int isLabel(char *line) {
    int len = strlen(line);
    if (len < 2) return 0; /* too short */
    if (line[len-1] != ':') return 0; /* must end with colon */
    for (int i = 0; i < len-1; i++)
        if (line[i]==' ' || line[i]=='\t') return 0;
    return 1; /* it is a label */
}
```

A label in MIPS assembly always ends with a colon and has no spaces in it, for example start: or loop:. A normal instruction like beq \$t0,\$t1,end is not a label even though it mentions the word end. This function checks three things: the line must be at least 2 characters, it must end with a colon, and it must not have spaces inside. If all three are true it returns 1 meaning yes this is a label, otherwise 0 meaning no.

### 3.6 getLabelAddress — Finding Label Addresses

```
int getLabelAddress(char *name) {
    char clean[64];
    strncpy(clean, name, 63);
    clean[63] = '\0';
    int l = strlen(clean);
    if (l>0 && clean[l-1]==':') clean[l-1]='\0';
    for (int i = 0; i < labelCount; i++)
        if (strcmp(labelTable[i].name, clean) == 0)
            return labelTable[i].address;
    return -1;
}
```

When the program sees `beq $t0,$t1,end` it needs to find what address the label `end` corresponds to. This function searches through the `labelTable` and returns the address. If the label is not found it returns -1. It also handles the case where the label name might accidentally have a colon at the end by removing it first.

### 3.7 parseMemOperand — Parsing Memory Addresses

```
int parseMemOperand(char *operand, int *offset) {
    char temp[128];
    strcpy(temp, operand);
    trim(temp);
    char *paren = strchr(temp, '(');
    if (!paren) {
        *offset = 0;
        return getRegNum(temp);
    }
    *paren = '\0';
    char *offsetStr = temp;
    char *regPart = paren + 1;
    char *close = strchr(regPart, ')');
    if (close) *close = '\0';
    trim(offsetStr); trim(regPart);
    *offset = (strlen(offsetStr)==0) ? 0 : atoi(offsetStr);
    return getRegNum(regPart);
}
```

Load word (`lw`) and store word (`sw`) instructions use a special format for memory addresses: `offset($register)`. For example `16($t0)` means address is value in `$t0` plus 16. Also `($sp)` means offset is 0 and register is `$sp`. This function splits the operand string at the opening bracket. Everything before the bracket is the offset and everything inside the brackets is the register name. It converts both and returns them.

### 3.8 main — File Reading

```
FILE *fp = fopen(argv[1], "r");
while (lineCount < MAX_LINES
    && fgets(lines[lineCount], MAX_LINE_LEN, fp)) {
    trim(lines[lineCount]);
    lineCount++;
}
fclose(fp);
```

The program takes the input filename from the command line as `argv[1]`. It opens the file and reads it line by line using `fgets`. Each line is trimmed to remove whitespace and stored in the `lines` array. All the actual work is done on the `lines` array not on the file directly.

### 3.9 Pass 1 — Building the Label Table

```
int addr = BASE_ADDRESS;
```



```

for (int i = 0; i < lineCount; i++) {
    if (isLabel(lines[i])) {
        /* store label name and address */
        strcpy(labelTable[labelCount].name, lines[i]);
        labelTable[labelCount].address = addr;
        labelCount++;
        /* address does NOT increase for labels */
    } else {
        addr += 4; /* each instruction = 4 bytes */
    }
}

```

In the first pass the program goes through every line. If a line is a label it stores the label name and current address in the labelTable. Notice the address does NOT increase for label lines because labels do not produce any machine code, they just mark the position of the next instruction. If a line is a normal instruction the address increases by 4.

```

Example: file contains "start:" then "add $t0,$t1,$t2"
=> label start gets address 0x00400000
=> add also gets address 0x00400000
=> because labels do not occupy any memory space

```

### 3.10 Pass 2 — Generating Machine Code

In the second pass the program reads each line again. Label lines are skipped. For instruction lines it extracts the mnemonic (instruction name) and operands by splitting the line at commas. For example `addi $t3,$t0,5` gives mnemonic=`addi`, op1=`$t3`, op2=`$t0`, op3=`5`. Then it builds the 32-bit machine code using bitwise shift operations.

### 3.11 R-Type Encoding — add, and, or, xor, slt

```

/* add $rd, $rs, $rt  funct = 0x20 */
int rd=getRegNum(op1), rs=getRegNum(op2), rt=getRegNum(op3);
machineCode=(0<<26)|(rs<<21)|(rt<<16)|(rd<<11)|(0<<6)|0x20;

```

For R-type instructions the 32 bits are built by combining six fields using bit shifting and OR operations. The `<<` symbol shifts bits left. For example `(rs<<21)` places the 5-bit register number into bits 25 to 21 of the 32-bit result. The `|` symbol combines all the fields together. The opcode is always 0 for R-type. The funct field goes in the lowest 6 bits with no shifting.

```

Verified example: add $9,$10,$11 => expected 0x014B4820
opcode = 000000  (6 bits, always 0 for R-type)
rs=$10  = 01010  (5 bits, register 10)
rt=$11  = 01011  (5 bits, register 11)
rd=$9   = 01001  (5 bits, register 9)
shamt   = 00000  (5 bits, always 0 here)

```

```

funct    = 100000 (6 bits, 0x20 = 32 for add)
Result   = 00000001010010110100100000100000 = 0x014B4820  MATCHES MARS

```

### 3.12 I-Type Encoding — addi, andi, ori

```

/* addi $rt, $rs, imm opcode = 0x08 */
int rt=getRegNum(op1), rs=getRegNum(op2), imm=atoi(op3);
machineCode=(0x08<<26)|(rs<<21)|(rt<<16)
            |((unsigned short)(short)imm);

```

For I-type instructions the opcode is in the top 6 bits, rs in bits 25-21, rt in bits 20-16, and the immediate value in the bottom 16 bits. The cast (unsigned short)(short)imm is important for negative numbers. When a number like -16 is stored it needs to be treated as a 16-bit two's complement value so the cast makes sure only 16 bits are used correctly.

```

Verified example: addi $12,$9,4 => expected 0x212C0004
opcode = 001000 (6 bits, 0x08 = 8 for addi)
rs=$9    = 01001 (5 bits, register 9)
rt=$12   = 01100 (5 bits, register 12)
imm=4     = 00000000000000100 (16 bits)
Result    = 001000010010110000000000000000100 = 0x212C0004  MATCHES MARS

```

### 3.13 BEQ — Branch on Equal

```

/* beq $rs, $rt, label opcode = 0x04 */
int rs=getRegNum(op1), rt=getRegNum(op2);
int labelAddr = getLabelAddress(op3);
int offset = (labelAddr - (addr + 4)) / 4;
machineCode=(0x04<<26)|(rs<<21)|(rt<<16)
            |((unsigned short)(short)offset);

```

BEQ is I-type but the immediate field stores a relative offset not a direct value. The offset tells the processor how many instructions forward or backward to jump. Adding 4 to the current address accounts for the fact that the PC has already moved to the next instruction. Dividing by 4 converts bytes to words since MIPS offset is word-relative.

```

Verified example: beq $12,$9,end
beq is at address 0x400008, end label is at 0x400014
offset = (0x400014 - (0x400008 + 4)) / 4
        = (0x400014 - 0x40000C) / 4
        = 8 / 4 = 2
Machine code = 0x11890002  MATCHES MARS

```

### 3.14 J — Jump Instruction

```
/* j label opcode = 0x02 */
int labelAddr = getLabelAddress(op1);
unsigned int target =
    ((unsigned int)labelAddr >> 2) & 0x03FFFFFF;
machineCode = (0x02 << 26) | target;
```

The j instruction uses J-type format. The 26-bit target field stores the word address of the destination. Shifting right by 2 converts byte address to word address. The mask 0x03FFFFFF keeps only the lower 26 bits which is all that fits in the target field.

```
Verified example: j start where start is at 0x00400000
target = 0x00400000 >> 2 = 0x00100000
masked = 0x00100000 AND 0x03FFFFFF = 0x00100000
Machine code = (0x02 << 26) | 0x00100000 = 0x08100000 MATCHES MARS
```

### 3.15 LW and SW — Memory Instructions

```
/* lw $rt, offset($rs) opcode = 0x23 */
int rt = getRegNum(op1);
int offset = 0;
int rs = parseMemOperand(op2, &offset);
machineCode=(0x23<<26)|(rs<<21)|(rt<<16)
            |((unsigned short)(short)offset);
```

Load word (lw) and store word (sw) are I-type instructions. They use the memory operand format offset(\$register). The parseMemOperand function extracts the offset number and register separately. After that the encoding is the same as other I-type instructions. LW has opcode 0x23 and SW has opcode 0x2B.

```
Verified example: sw $t0,0($sp)
opcode=0x2B, rs=$sp=29, rt=$t0=8, offset=0
Machine code = 0xAFA80000 MATCHES MARS
```

### 3.16 Final Output Line

```
printf("0x%08X 0x%08X %s\n",
        addr, machineCode, original);
addr += 4;
```

After calculating machine code the program prints three things on one line. First the current memory address in 8-digit uppercase hex. Second, the machine code in 8-digit uppercase hex. Third, the original assembly instruction exactly as it was written in the input file. Then the address increases by 4 to prepare for the next instruction.

## Results

The assembler was tested with six input files. All outputs were verified against the MARS MIPS simulator. The terminal screenshots show the program output and the MARS snapshots confirm the machine code values are correct.

### Input File 1 — in1.txt

This file tests the basic R-type instructions `add` and `and`, along with I-type instructions `addi` and `or`. The output shows correct machine code for all six instructions starting from address `0x00400000`.

Instructions tested: `add`, `addi`, `and`, `or`

*in1.txt — Terminal Output Screenshot*

```
obil@ubuntu:~/Desktop/Compiler A-2$ ./test in1.txt
Assembly language program:
add $t0,$t1,$t2
addi $t3,$t0,5
and $t4,$t0,$t3
or $t5,$t4,$t3
addi $t6,$zero,10
add $t7,$t5,$t6

Machine Code:
0x00400000 0x012A4020 add $t0,$t1,$t2
0x00400004 0x210B0005 addi $t3,$t0,5
0x00400008 0x010B6024 and $t4,$t0,$t3
0x0040000C 0x018B6825 or $t5,$t4,$t3
0x00400010 0x200E000A addi $t6,$zero,10
0x00400014 0x01AE7820 add $t7,$t5,$t6
```

*snap1 — MARS Text Segment for in1.asm*

| Text Segment             |            |            |                          |        |                     |
|--------------------------|------------|------------|--------------------------|--------|---------------------|
| Bkpt                     | Address    | Code       | Basic                    | Source |                     |
| <input type="checkbox"/> | 0x00400000 | 0x012a4020 | add \$8,\$9,\$10         | 1:     | add \$t0,\$t1,\$t2  |
| <input type="checkbox"/> | 0x00400004 | 0x210b0005 | addi \$11,\$8,0x00000005 | 2:     | addi \$t3,\$t0,5    |
| <input type="checkbox"/> | 0x00400008 | 0x010b6024 | and \$12,\$8,\$11        | 3:     | and \$t4,\$t0,\$t3  |
| <input type="checkbox"/> | 0x0040000c | 0x018b6825 | or \$13,\$12,\$11        | 4:     | or \$t5,\$t4,\$t3   |
| <input type="checkbox"/> | 0x00400010 | 0x200e000a | addi \$14,\$0,0x0000000a | 5:     | addi \$t6,\$zero,10 |
| <input type="checkbox"/> | 0x00400014 | 0x01ae7820 | add \$15,\$13,\$14       | 6:     | add \$t7,\$t5,\$t6  |

## Input File 2 — in2.txt

This file tests the immediate instructions `andi` and `ori`, along with R-type instructions `xor` and `slt`. All six instructions were correctly translated and the hex values match the MARS simulator output.

Instructions tested: `addi`, `andi`, `ori`, `xor`, `slt`

*in2.txt — Terminal Output Screenshot*

```
obil@ubuntu:~/Desktop/Compiler A-2$ ./test in2.txt
Assembly language program:
addi $s0,$zero,15
andi $s1,$s0,7
ori $s2,$s0,3
xor $s3,$s1,$s2
slt $s4,$s1,$s2
andi $s5,$s4,1

Machine Code:
0x00400000 0x2010000f addi $s0,$zero,15
0x00400004 0x32110007 andi $s1,$s0,7
0x00400008 0x36120003 ori $s2,$s0,3
0x0040000c 0x02329826 xor $s3,$s1,$s2
0x00400010 0x0232A02A slt $s4,$s1,$s2
0x00400014 0x32950001 andi $s5,$s4,1
```

*snap2 — MARS Text Segment for in2.asm*

| Address    | Code       | Basic                    | Source                 |
|------------|------------|--------------------------|------------------------|
| 0x00400000 | 0x2010000f | addi \$16,\$0,0x0000000f | 1: addi \$s0,\$zero,15 |
| 0x00400004 | 0x32110007 | andi \$17,\$16,0x0000... | 2: andi \$s1,\$s0,7    |
| 0x00400008 | 0x36120003 | ori \$18,\$16,0x00000003 | 3: ori \$s2,\$s0,3     |
| 0x0040000c | 0x02329826 | xor \$19,\$17,\$18       | 4: xor \$s3,\$s1,\$s2  |
| 0x00400010 | 0x0232a02a | slt \$20,\$17,\$18       | 5: slt \$s4,\$s1,\$s2  |
| 0x00400014 | 0x32950001 | andi \$21,\$20,0x0000... | 6: andi \$s5,\$s4,1    |

### Input File 3 — in3.txt

This file tests the memory instructions sw and lw which use the offset(\$register) format, along with addi and add. The assembler correctly parsed the memory operands and produced the right machine code for all six instructions.

Instructions tested: addi, sw, lw, add

*in3.txt — Terminal Output Screenshot*

```
obil@ubuntu:~/Desktop/Compiler A-2$ ./test in3.txt
Assembly language program:
addi $sp,$sp,-16
sw $t0,0($sp)
sw $t1,4($sp)
lw $t2,0($sp)
lw $t3,4($sp)
add $t4,$t2,$t3

Machine Code:
0x00400000 0x23BDDFF0 addi $sp,$sp,-16
0x00400004 0xAFA80000 sw $t0,0($sp)
0x00400008 0xAFA90004 sw $t1,4($sp)
0x0040000C 0x8FAA0000 lw $t2,0($sp)
0x00400010 0x8FAB0004 lw $t3,4($sp)
0x00400014 0x014B6020 add $t4,$t2,$t3
```

*snap3 — MARS Text Segment for in3.asm*

| Address    | Code       | Basic                    | Source                |
|------------|------------|--------------------------|-----------------------|
| 0x00400000 | 0x23bdfff0 | addi \$29,\$29,0xffff... | 1: addi \$sp,\$sp,-16 |
| 0x00400004 | 0xafa80000 | sw \$8,0x00000000(\$29)  | 2: sw \$t0,0(\$sp)    |
| 0x00400008 | 0xafa90004 | sw \$9,0x00000004(\$29)  | 3: sw \$t1,4(\$sp)    |
| 0x0040000c | 0x8faa0000 | lw \$10,0x00000000(\$29) | 4: lw \$t2,0(\$sp)    |
| 0x00400010 | 0x8fab0004 | lw \$11,0x00000004(\$29) | 5: lw \$t3,4(\$sp)    |
| 0x00400014 | 0x014b6020 | add \$12,\$10,\$11       | 6: add \$t4,\$t2,\$t3 |

## Input File 4 — in4.txt

This file tests the branch instruction beq and the jump instruction j along with addi, add, and and. Three labels are used which are start, loop, and done. The branch offset and jump target were both calculated correctly.

Instructions tested: addi, beq, add, j, and — uses labels start, loop, done

*in4.txt — Terminal Output Screenshot*

```
obil@ubuntu:~/Desktop/Compiler A-2$ ./test in4.txt
Assembly language program:
start:
addi $t0,$zero,0
addi $t1,$zero,5
loop:
beq $t0,$t1,done
add $t2,$t0,$t1
addi $t0,$t0,1
j loop
done:
and $t3,$t2,$t1

Machine Code:
0x00400000 0x20080000 addi $t0,$zero,0
0x00400004 0x20090005 addi $t1,$zero,5
0x00400008 0x11090003 beq $t0,$t1,done
0x0040000C 0x01095020 add $t2,$t0,$t1
0x00400010 0x21080001 addi $t0,$t0,1
0x00400014 0x08100002 j loop
0x00400018 0x01495824 and $t3,$t2,$t1
```

*snap4 — MARS Text Segment for in4.asm*

| Address    | Code       | Basic                   | Source                 |
|------------|------------|-------------------------|------------------------|
| 0x00400000 | 0x20080000 | addi \$8,\$0,0x00000000 | 2: addi \$t0,\$zero,0  |
| 0x00400004 | 0x20090005 | addi \$9,\$0,0x00000005 | 3: addi \$t1,\$zero,5  |
| 0x00400008 | 0x11090003 | beq \$8,\$9,0x00000003  | 5: beq \$t0,\$t1,done  |
| 0x0040000c | 0x01095020 | add \$10,\$8,\$9        | 6: add \$t2,\$t0,\$t1  |
| 0x00400010 | 0x21080001 | addi \$8,\$8,0x00000001 | 7: addi \$t0,\$t0,1    |
| 0x00400014 | 0x08100002 | j 0x00400008            | 8: j loop              |
| 0x00400018 | 0x01495824 | and \$11,\$10,\$9       | 10: and \$t3,\$t2,\$t1 |

## Input File 5 — in5.txt

This file tests eight instructions together including ori, xor, slt, beq, or, j, and andi. Two labels begin and skip are used. The backward jump using j and the forward branch using beq both produced correct offsets.

Instructions tested: addi, ori, xor, slt, beq, or, j, andi — uses labels begin, skip

*in5.txt — Terminal Output Screenshot*

```
obil@ubuntu:~/Desktop/Compiler A-2$ ./test in5.txt
Assembly language program:
begin:
addi $s0,$zero,4
ori $s1,$zero,2
xor $s2,$s0,$s1
slt $s3,$s1,$s0
beq $s3,$zero,skip
or $s4,$s0,$s1
j begin
skip:
andi $s5,$s0,3

Machine Code:
0x00400000 0x20100004 addi $s0,$zero,4
0x00400004 0x34110002 ori $s1,$zero,2
0x00400008 0x02119026 xor $s2,$s0,$s1
0x0040000C 0x0230982A slt $s3,$s1,$s0
0x00400010 0x12600002 beq $s3,$zero,skip
0x00400014 0x0211A025 or $s4,$s0,$s1
0x00400018 0x08100000 j begin
0x0040001C 0x32150003 andi $s5,$s0,3
```

*snap5 — MARS Text Segment for in5.asm*

| Address    | Code       | Basic                    | Source                  |
|------------|------------|--------------------------|-------------------------|
| 0x00400000 | 0x20100004 | addi \$16,\$0,0x00000004 | 2: addi \$s0,\$zero,4   |
| 0x00400004 | 0x34110002 | ori \$17,\$0,0x00000002  | 3: ori \$s1,\$zero,2    |
| 0x00400008 | 0x02119026 | xor \$18,\$16,\$17       | 4: xor \$s2,\$s0,\$s1   |
| 0x0040000c | 0x0230982a | slt \$19,\$17,\$16       | 5: slt \$s3,\$s1,\$s0   |
| 0x00400010 | 0x12600002 | beq \$19,\$0,0x00000002  | 6: beq \$s3,\$zero,skip |
| 0x00400014 | 0x0211a025 | or \$20,\$16,\$17        | 7: or \$s4,\$s0,\$s1    |
| 0x00400018 | 0x08100000 | j 0x00400000             | 8: j begin              |
| 0x0040001c | 0x32150003 | andi \$21,\$16,0x0000... | 10: andi \$s5,\$s0,3    |



### Input File 6 — in6.txt

This file contains all twelve instructions in one program and is the most complete test case. It uses labels main and end. All fifteen instruction lines were correctly translated and the output was split into two screenshots because it was too long to fit in one screen.

Instructions tested: all twelve instructions together — uses labels main, end

*in6.txt — Terminal Output Screenshot (First Half)*

```
obil@ubuntu:~/Desktop/Compiler A-2$ ./test in6.txt
Assembly language program:
main:
addi $t0,$zero,8
addi $t1,$zero,3
add $t2,$t0,$t1
and $t3,$t0,$t1
or $t4,$t0,$t1
xor $t5,$t0,$t1
slt $t6,$t1,$t0
andi $t7,$t0,5
ori $s0,$t1,4
addi $sp,$sp,-8
sw $t2,0($sp)
lw $s1,0($sp)
beq $t6,$zero,end
j main
end:
add $s2,$s0,$s1
```

*in6.txt — Terminal Output Screenshot (Second Half)*

```
Machine Code:
0x00400000 0x20080008 addi $t0,$zero,8
0x00400004 0x20090003 addi $t1,$zero,3
0x00400008 0x01095020 add $t2,$t0,$t1
0x0040000C 0x01095824 and $t3,$t0,$t1
0x00400010 0x01096025 or $t4,$t0,$t1
0x00400014 0x01096826 xor $t5,$t0,$t1
0x00400018 0x0128702A slt $t6,$t1,$t0
0x0040001C 0x310F0005 andi $t7,$t0,5
0x00400020 0x35300004 ori $s0,$t1,4
0x00400024 0x23BDFFF8 addi $sp,$sp,-8
0x00400028 0xAFAA0000 sw $t2,0($sp)
0x0040002C 0x8FB10000 lw $s1,0($sp)
0x00400030 0x11C00001 beq $t6,$zero,end
0x00400034 0x08100000 j main
0x00400038 0x02119020 add $s2,$s0,$s1
```

*snap6 — MARS Text Segment for in6.asm*

| Address    | Code       | Basic                    | Source                  |
|------------|------------|--------------------------|-------------------------|
| 0x00400000 | 0x20080008 | addi \$8,\$0,0x00000008  | 2: addi \$t0,\$zero,8   |
| 0x00400004 | 0x20090003 | addi \$9,\$0,0x00000003  | 3: addi \$t1,\$zero,3   |
| 0x00400008 | 0x01095020 | add \$10,\$8,\$9         | 4: add \$t2,\$t0,\$t1   |
| 0x0040000c | 0x01095824 | and \$11,\$8,\$9         | 5: and \$t3,\$t0,\$t1   |
| 0x00400010 | 0x01096025 | or \$12,\$8,\$9          | 6: or \$t4,\$t0,\$t1    |
| 0x00400014 | 0x01096826 | xor \$13,\$8,\$9         | 7: xor \$t5,\$t0,\$t1   |
| 0x00400018 | 0x0128702a | slt \$14,\$9,\$8         | 8: slt \$t6,\$t1,\$t0   |
| 0x0040001c | 0x310f0005 | andi \$15,\$8,0x00000005 | 9: andi \$t7,\$t0,5     |
| 0x00400020 | 0x35300004 | ori \$16,\$9,0x00000004  | 10: ori \$s0,\$t1,4     |
| 0x00400024 | 0x23bdfff8 | addi \$29,\$29,0xffff... | 11: addi \$sp,\$sp,-8   |
| 0x00400028 | 0xafaa0000 | sw \$10,0x00000000(\$29) | 12: sw \$t2,0(\$sp)     |
| 0x0040002c | 0x8fb10000 | lw \$17,0x00000000(\$29) | 13: lw \$s1,0(\$sp)     |
| 0x00400030 | 0x11c00001 | beq \$14,\$0,0x00000001  | 14: beq \$t6,\$zero,end |
| 0x00400034 | 0x08100000 | j 0x00400000             | 15: j main              |
| 0x00400038 | 0x02119020 | add \$18,\$16,\$17       | 17: add \$s2,\$s0,\$s1  |

## Instruction Coverage

The table below shows each instruction appears in at least 2 input files as required by the assignment rubric.

| Instruction | Type   | Appears In                   | Files   |
|-------------|--------|------------------------------|---------|
| add         | R-type | in1, in3, in4, in6           | 4 files |
| and         | R-type | in1, in4, in6                | 3 files |
| or          | R-type | in1, in5, in6                | 3 files |
| xor         | R-type | in2, in5, in6                | 3 files |
| slt         | R-type | in2, in5, in6                | 3 files |
| addi        | I-type | in1, in2, in3, in4, in5, in6 | 6 files |
| andi        | I-type | in2, in5, in6                | 3 files |
| ori         | I-type | in2, in5, in6                | 3 files |
| lw          | I-type | in3, in6                     | 2 files |
| sw          | I-type | in3, in6                     | 2 files |
| beq         | I-type | in4, in5, in6                | 3 files |
| j           | J-type | in4, in5, in6                | 3 files |

## Discussion

The assembler produced correct results for all six input files. Every machine code value matched the MARS simulator output exactly. This shows that the bit encoding logic in the code is implemented correctly.

The most difficult part of this project was the beq instruction. The branch offset formula (label\_address minus (PC plus 4)) divided by 4 was confusing at first because it was not obvious why we need to add 4 to the current address. After understanding how MIPS works it became clear that the processor always increments the PC by 4 before checking the branch condition so the offset is always relative to the next instruction not the current one.

Another challenge was the two pass approach. At first it seemed like the program could just go through the file one time. But then came the realization that if a beq instruction at the beginning of the file jumps to a label at the end of the file, the offset cannot be calculated in one pass

because the label has not been seen yet. This is called a forward reference. The two pass solution fixes this completely.

The `parseMemOperand` function was also tricky because memory operands can appear in different forms like `16($t0)`, `0($sp)`, or `($t2)` with no offset at all. The function handles all these cases by checking if the offset part is empty and using zero in that case.

One small cosmetic difference between our output and MARS is that our program uses uppercase hex letters like `0x012A4020` while MARS uses lowercase like `0x012a4020`. These represent exactly the same number. Hexadecimal letters are not case sensitive so this does not affect correctness at all.

A possible improvement for the future would be to add more instructions like `sub`, `mult`, `div` and system calls. Also better error messages could be added so that if the input file has a mistake the program gives a clear explanation of what went wrong instead of producing wrong output.

## Conclusions

In this assignment, a working MIPS assembler was built using the C programming language. The assembler correctly translates twelve MIPS assembly instructions into their 32-bit hexadecimal machine code. The program was designed using the standard two pass assembly technique where the first pass collects label addresses and the second pass generates machine code.

The main research question of this project was how a C program can correctly translate MIPS assembly instructions into binary machine code. This question was answered by implementing the correct bitwise encoding for all three MIPS instruction types: R-type which uses opcode 0 with a funct field, I-type which uses a specific opcode with a 16-bit immediate field and J-type which encodes a 26-bit word address.

All six input files were tested and every output matched the MARS MIPS simulator output perfectly. This project gave a very clear understanding of how assembly language is connected to machine code and why assemblers are an important tool in the chain from source code to executable programs. It also showed the importance of binary number representation and bit manipulation which are fundamental skills in low level programming and compiler construction.

## AI Assistance

This section documents the use of AI-based tools to support the development of this project. The assistance was used for understanding concepts, getting help with code structure and verifying that the logic was correct. All outputs were carefully reviewed, tested against the MARS MIPS simulator and modified where necessary to ensure correctness and alignment with the assignment requirements.

Prompts used during the project:

- "How do I encode R-type, I-type and J-type MIPS instructions into 32-bit machine code using bit shifting in C?"
- "What types of MIPS instructions should be included in test files to properly cover all instruction formats?"
- "How do I calculate the branch offset for a beq instruction using PC-relative addressing in MIPS?"
- "How can I implement a two-pass assembler in C to handle labels and forward references?"
- "How do I compute the jump target address for a MIPS j instruction and store it in the 26-bit field?"
- "How do I parse memory operands like offset(\$register) in C for lw and sw instructions?"

## References

Ricroc, "Encoding MIPS Instructions," Department of Computer Science, University of Porto, Portugal, 2019-2020.

Available: [https://www.dcc.fc.up.pt/~ricroc/aulas/1920/ac/apontamentos/P04\\_encoding\\_mips\\_instructions.pdf](https://www.dcc.fc.up.pt/~ricroc/aulas/1920/ac/apontamentos/P04_encoding_mips_instructions.pdf)

A. Max, "MIPS Machine Language Instruction Formats," Kalamazoo College.

Available: <https://max.cs.kzoo.edu/cs230/Resources/MIPS/MachineXL/InstructionFormats.html>

Harvard EECS, "MIPS Architecture Part I," Harvard University.

Available: <https://www.eecs.harvard.edu/~cs161/notes/mips-part-I.pdf>

K. Vollmar and P. Sanderson, "MARS: MIPS Assembler and Runtime Simulator," Missouri State University. [Online].

Available: <https://courses.missouristate.edu/KenVollmar/MARS/>